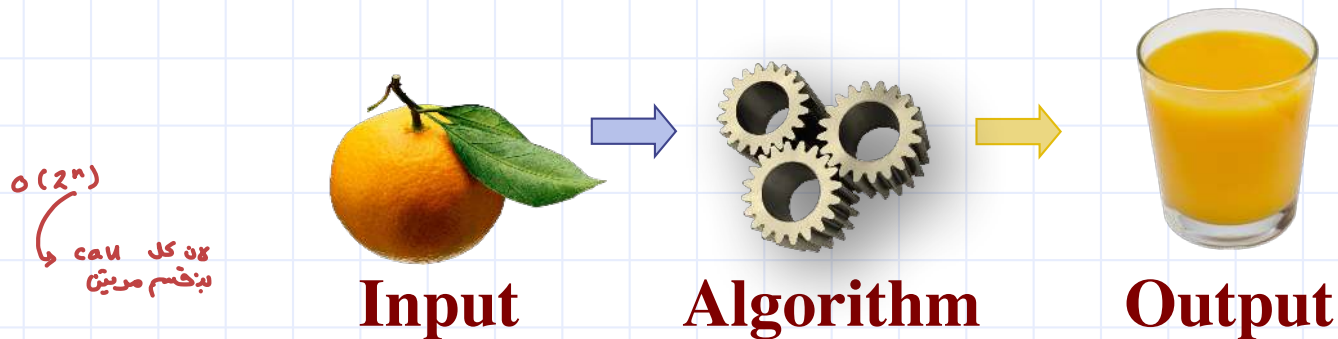


Presentation for use with the textbook **Data Structures and Algorithms in Java, 6th edition**, by M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, Wiley, 2014

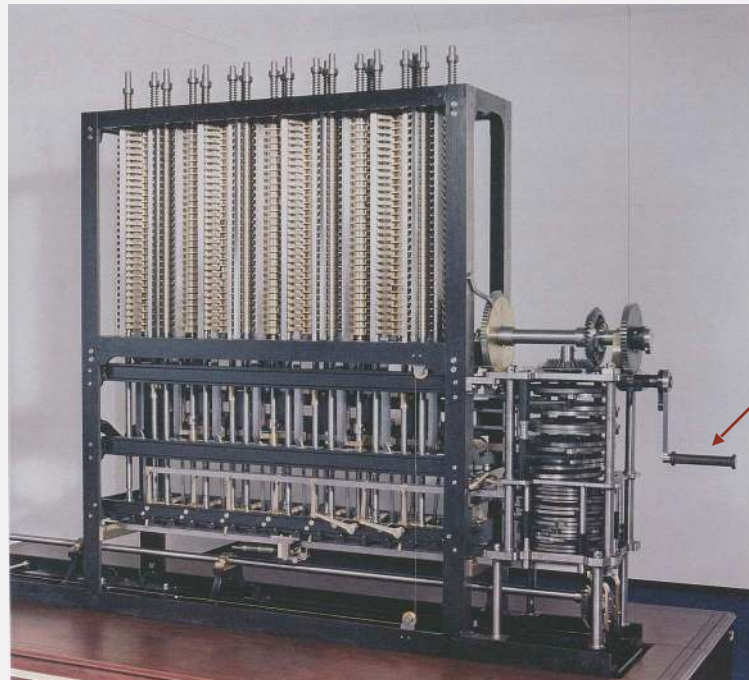
Analysis of Algorithms

*important chapter



Running time

“ As soon as an Analytic Engine exists, it will necessarily guide the future course of the science. Whenever any result is sought by its aid, the question will arise—By what course of calculation can these results be arrived at by the machine in the shortest time? ” — Charles Babbage (1864)



how many times do you
have to turn the crank?

Analytic Engine

Measuring the running time

Q. How to time a program?

A. Manual.



```
% java ThreeSum 1Kints.txt
```



tick tick tick

70

```
% java ThreeSum 2Kints.txt
```

[illegible]

528

```
% java ThreeSum 4Kints.txt
```

[illegible]

4039

Measuring the running time

Q. How to time a program?

A. Automatic.

<code>public class Stopwatch</code>	<code>(part of stdlib.jar)</code>
<code>Stopwatch()</code>	<i>create a new stopwatch</i>
<code>double elapsedTime()</code>	<i>time since creation (in seconds)</i>

```
public static void main(String[] args)
{
    In in = new In(args[0]);
    int[] a = in.readAllInts();
    Stopwatch stopwatch = new Stopwatch();
    StdOut.println(ThreeSum.count(a));
    double time = stopwatch.elapsedTime();
    StdOut.println("elapsed time " + time);
}
```

Measuring the running time

Q. How to time a program?

A. Automatic.

```
public class Stopwatch
```

```
    Stopwatch()
```

create a new stopwatch

```
    double elapsedTime()
```

time since creation (in seconds)

```
public class Stopwatch
{
    private final long start = System.currentTimeMillis();

    public double elapsedTime()
    {
        long now = System.currentTimeMillis();
        return (now - start) / 1000.0;
    }
}
```

implementation (part of stdlib.jar)

```
public static String repeat1(char c, int n) {  
    String answer = "";  
    for (int j=0; j < n; j++)  
        answer += c;  
    return answer;  
}
```

→ begin

→ end time

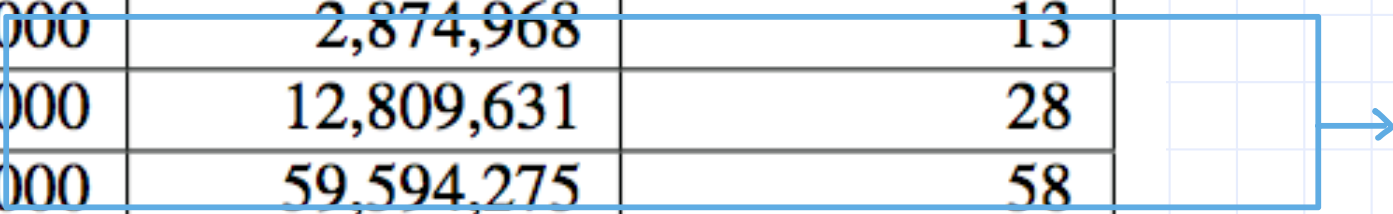
end - beg

```
public static String repeat2(char c, int n) {  
    StringBuilder sb = new StringBuilder();  
    for (int j=0; j < n; j++)  
        sb.append(c);  
    return sb.toString();  
}
```

→ جيداً أسرع

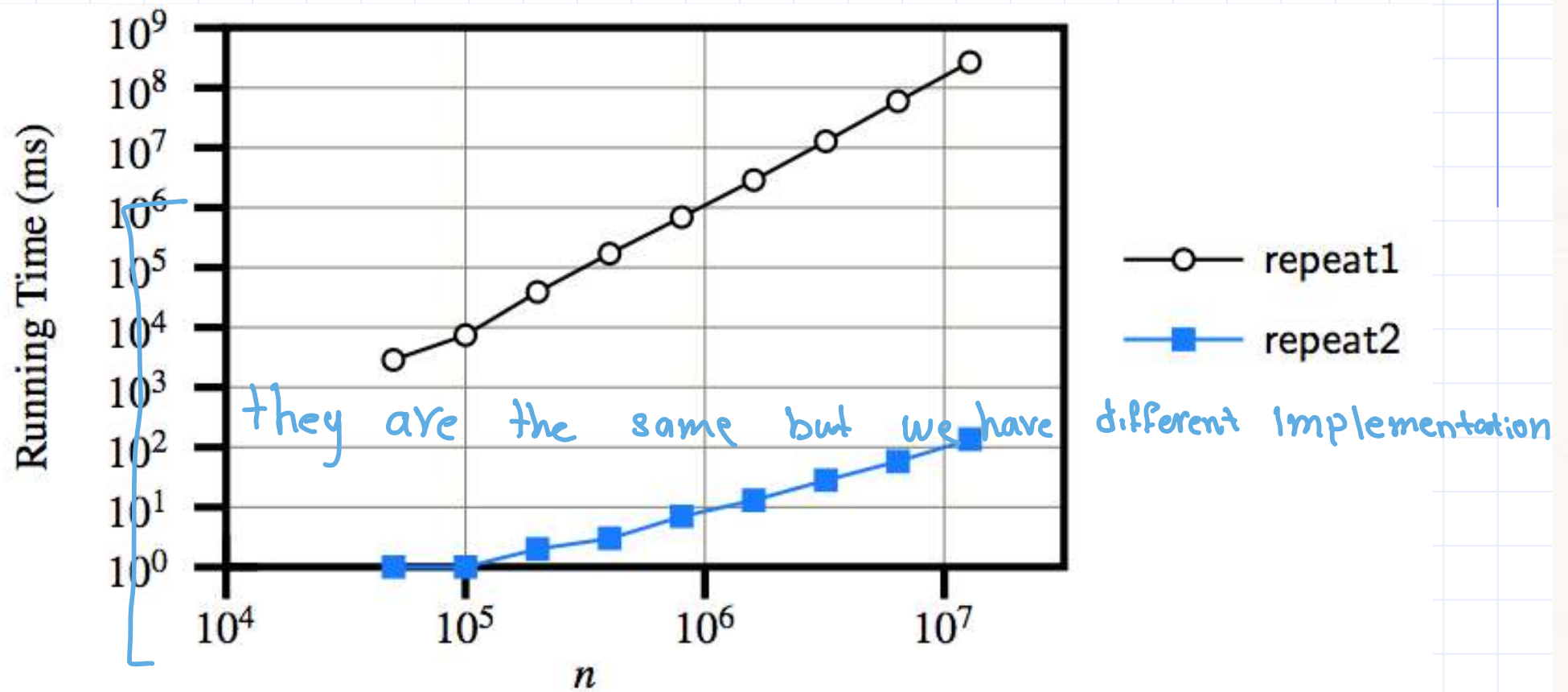


n	repeat1 (in ms)	repeat2 (in ms)
50,000	2,884	1
100,000	7,437	1
200,000	39,158	2
400,000	170,173	3
800,000	690,836	7
1,600,000	2,874,968	13
3,200,000	12,809,631	28
6,400,000	59,594,275	58
12,800,000	265,696,421	135



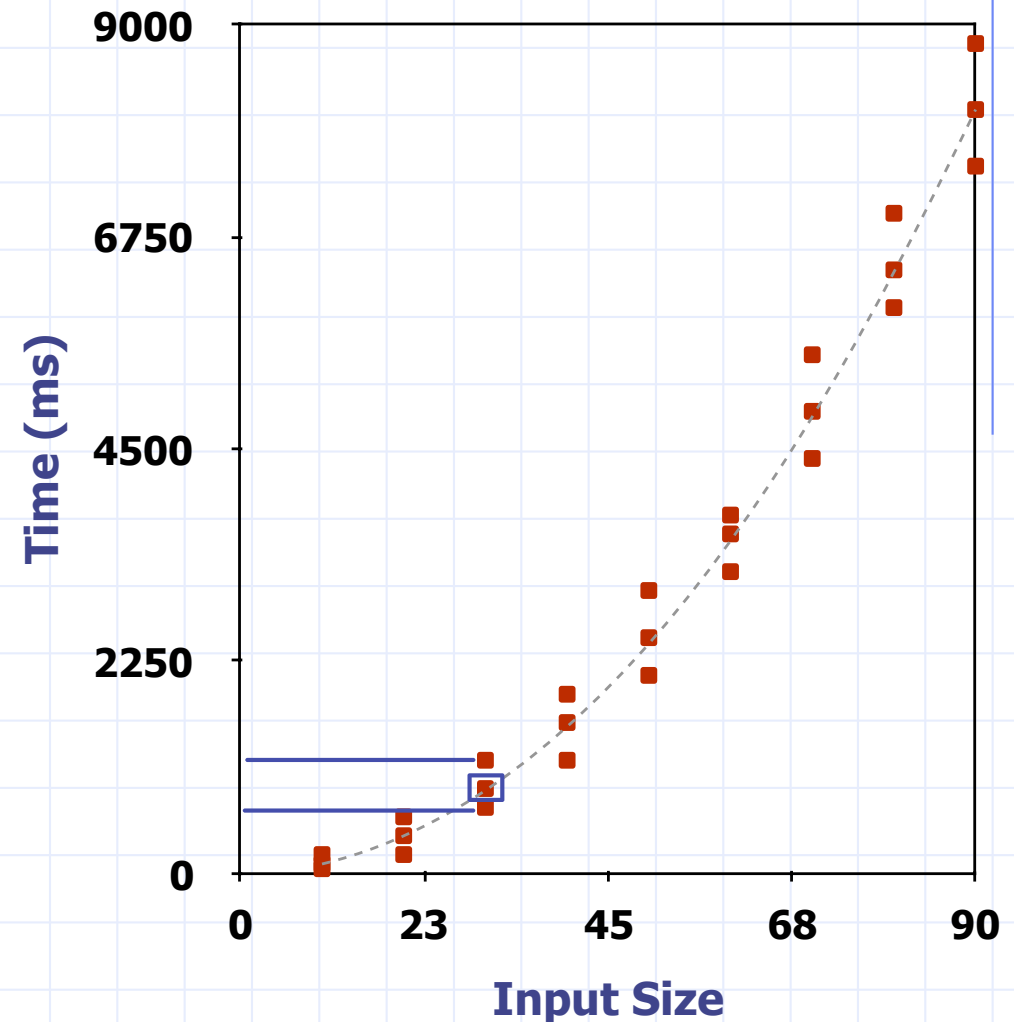
this
to

→ 2 → 4



Experimental Studies

- Write a program implementing the algorithm
- Run the program with inputs of varying size and composition, noting the time needed:
- Plot the results



```
1 long startTime = System.currentTimeMillis();
2 /* (run the algorithm) */
3 long endTime = System.currentTimeMillis();
4 long elapsed = endTime - startTime;
```

```
// record the starting time
```

```
// record the ending time
```

```
// compute the elapsed time
```

Limitations of Experiments

- ❑ It is necessary to implement the algorithm, which may be difficult
- ❑ Results may not be indicative of the running time on other inputs not included in the experiment.
- ❑ In order to compare two algorithms, the same hardware and software environments must be used



Challenges

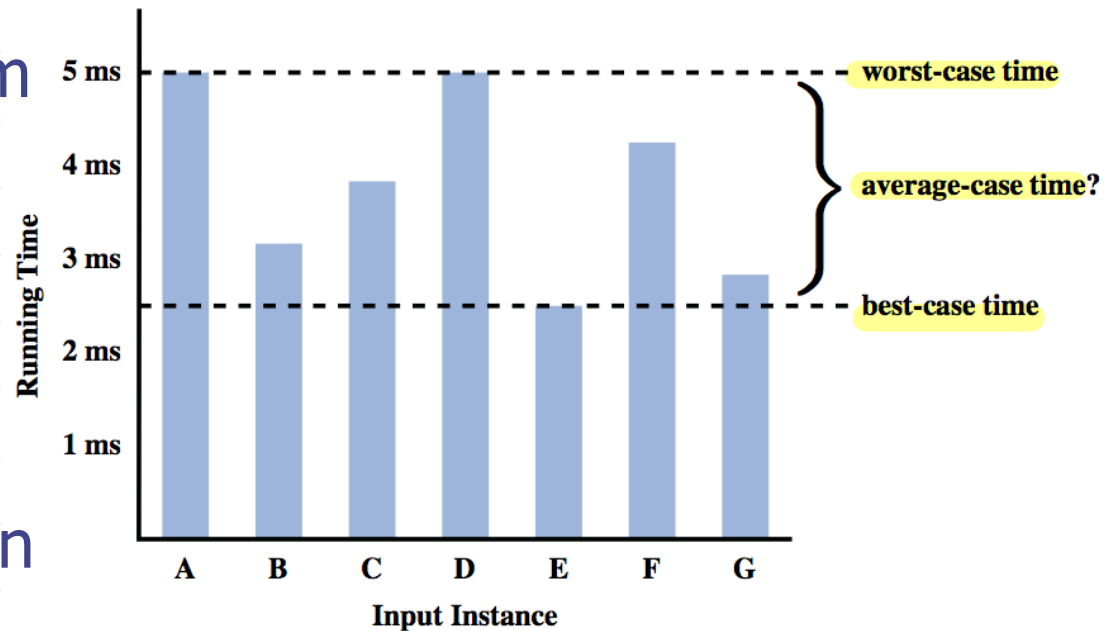
- Experimental running times of two algorithms are difficult to directly compare unless the experiments are performed in the same hardware and software environments.
- Experiments can be done only on a limited set of test inputs; hence, they leave out the running times of inputs not included in the experiment (and these inputs may be important).
- An algorithm must be fully implemented in order to execute it to study its running time experimentally.

Goals of Algorithm analysis

1. Allows us to evaluate the relative efficiency of any two algorithms in a way that is independent of the hardware and software environment.
2. Is performed by studying a high-level description of the algorithm without need for implementation.
3. Takes into account all possible inputs.

Running Time

- ❑ Most algorithms transform input objects into output objects.
- ❑ The running time of an algorithm typically grows with the input size.
- ❑ Average case time is often difficult to determine.
- ❑ We focus on the worst case running time.
 - Easier to analyze
 - Crucial to applications such as games, finance and robotics



Theoretical Analysis



- مايعتمد على لغة معينة
❑ Uses a high-level description of the algorithm instead of an implementation
- ❑ Characterizes running time as a function of the input size, n
- ❑ Takes into account all possible inputs
- ❑ Allows us to evaluate the speed of an algorithm independent of the hardware/software environment

Primitive Operations



- Basic computations performed by an algorithm
- Identifiable in pseudocode
- Largely independent from the programming language
- Exact definition not important (we will see why later)
- Assumed to take a constant amount of time in the RAM model

Examples:

- Evaluating an expression
- Assigning a value to a variable =
- Indexing into an array []
- Calling a method
- Returning from a method

n $\{ \text{for } (i \leftarrow 1 \text{ to } n) \text{ do } \dots \}$ التكرار $\rightarrow 3$

عندما جديده ونهاية

المساواة تنحسب .

إذا ما قيمة مساواة معناته صوابا يوقف اللوب .

الاقولس المربعة .

(الشرط التوقف)

Counting Primitive Operations

- By inspecting the pseudocode, we can determine the maximum number of primitive operations executed by an algorithm, as a function of the input size

```

1  /** Returns the maximum value of a nonempty array of numbers. */
2  public static double arrayMax(double[ ] data) {
3      int n = data.length;
4      double currentMax = data[0];           // assume first entry is biggest (for now)
5      for (int j = 1; j <= n; j++)           // consider all other entries
6          if (data[j] > currentMax)           // if data[j] is biggest thus far...
7              currentMax = data[j];           // record it as the current max
8      return currentMax;
9  }
```

$$2 + 2(n-1) + 2(n-1) + 1$$

$$2 + 2n - 2 + 2n - 2 + 1 = 4n + 3$$

- Step 3: 2 ops,
- 5: $2n$ ops,
- 7: 0 to n ops,

- 4: 2 ops,
- 6: $2n$ ops,
- 8: 1 op

Ex :

n 1- for($\leftarrow$ to < 3) 1, 2, 3^x 3 $\rightarrow$ عدد التكرار

نماذج for (1 to <=3) 1, 2, 3, 4 4 ← عدد التكرار
 أتم العمليات تزايد بواحد .

n4:3 → صناما یشتل
for (0 to < 3) → 0, 1, 2, 3^x
→ بونیه مغنی ورتیله = من قدام.

$n+2$ 4- $\text{for}(\underline{0} \text{ to } \leq \underline{3}) \rightarrow \underline{0}, 1, 2, 3, \underline{4}^x$

Body of the loop: → البودي كم مره اشتغل

1 - n - 1 →

2 ٧١ → **الابوة بتعمل التستيجب**
ما راح تشتغل + فية معاها

Cost of basic operations



Challenge. How to estimate constants.

operation	example	nanoseconds [†]
integer add	$a + b$	2.1
integer multiply	$a * b$	2.4
integer divide	a / b	5.4
floating-point add	$a + b$	4.6
floating-point multiply	$a * b$	4.2
floating-point divide	a / b	13.5
sine	<code>Math.sin(theta)</code>	91.3
arctangent	<code>Math.atan2(y, x)</code>	129
...	...	...

[†] Running OS X on Macbook Pro 2.2GHz with 2GB RAM

Cost of basic operations

Observation. Most primitive operations take constant time.

 $\rightarrow$ WCS

operation	example	nanoseconds [†]
variable declaration	<code>int a</code>	c_1
assignment statement	<code>a = b</code>	c_2
integer compare	<code>a < b</code>	c_3
array element access	<code>a[i]</code>	c_4
array length	<code>a.length</code>	c_5
1D array allocation	<code>new int[N]</code>	$c_6 N$
2D array allocation	<code>new int[N][N]</code>	$c_7 N^2$

Caveat. Non-primitive operations often take more than constant time.


novice mistake: abusive string concatenation

Example: 1-SUM

Q. How many instructions as a function of input size N ?

```
int count = 0;
for (int i = 0; i < N; i++)
    if (a[i] == 0)
        count++;
```

Annotations:

- int count = 0;**: variable declaration (2 instructions)
- for (int i = 0; i < N; i++)**: assignment statement (2 instructions), less than compare ($N + 1$), increment (N to $2N$)
- if (a[i] == 0)**: array access (N), equal to compare (N)
- count++;**: increment (N to $2N$)

Diagram notes:

- Increment inside the loop**: points to $i++$
- N array accesses**: points to $a[i]$
- assignment**: points to $i = 0$
- Compare**: points to $i < N$

operation	frequency
variable declaration	<u>2</u>
assignment statement	2
less than compare	$N + 1$
equal to compare	N
array access	N
increment	N to $2N$

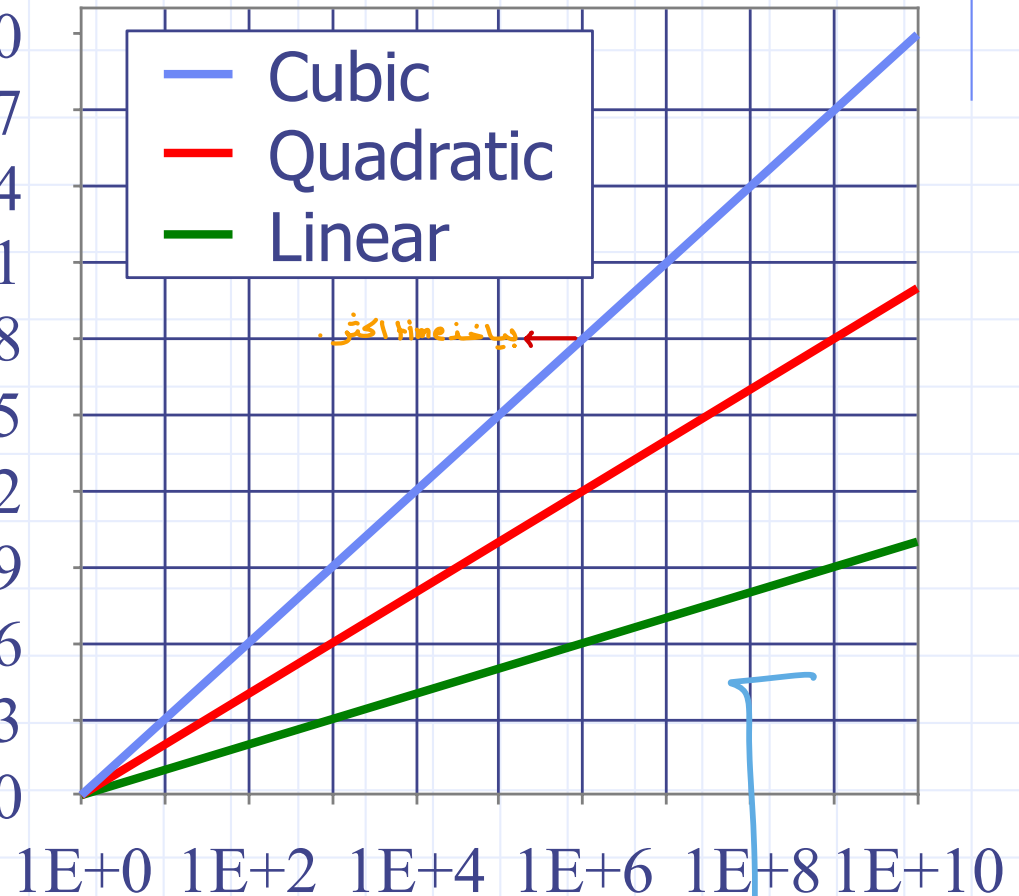
Seven Important Functions

- Seven functions that often appear in algorithm analysis:

- Constant ≈ 1 Binary Search
- Logarithmic $\approx \log n$ ←
- Linear $\approx n$ ← اقل time
- N-Log-N $\approx n \log n$ ← merge Sort
- Quadratic $\approx n^2$
- Cubic $\approx n^3$
- Exponential $\approx 2^n$ ← اكثر واحد بال time Slope

$T(n)$

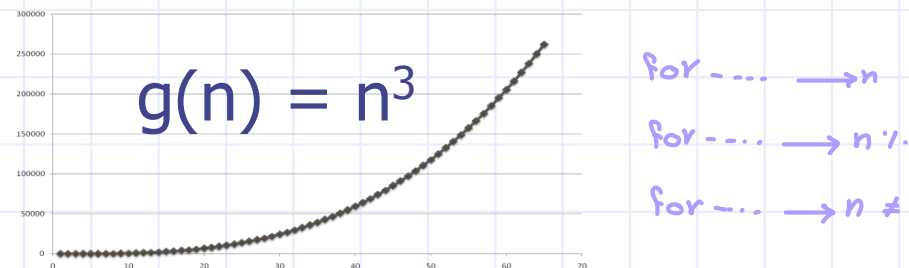
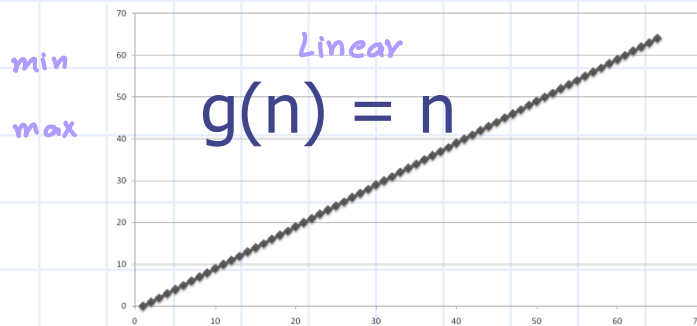
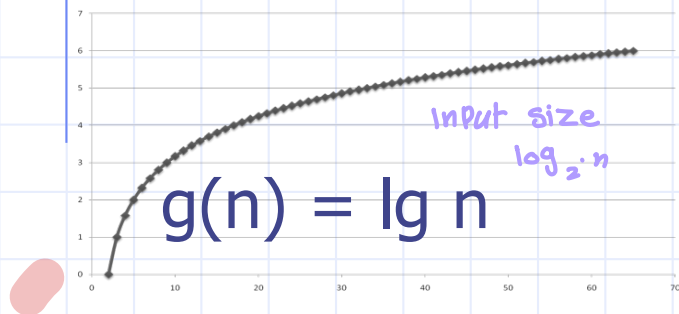
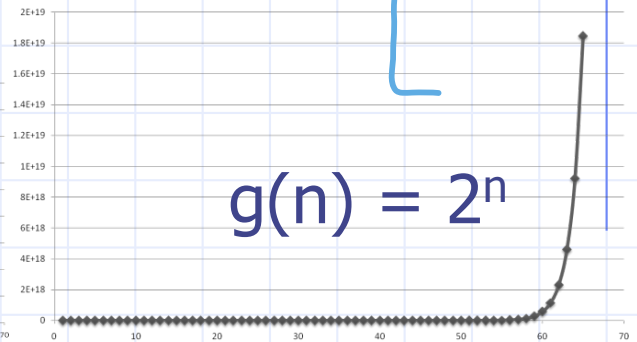
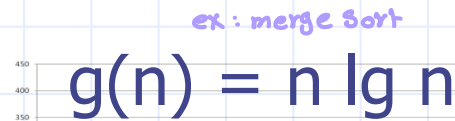
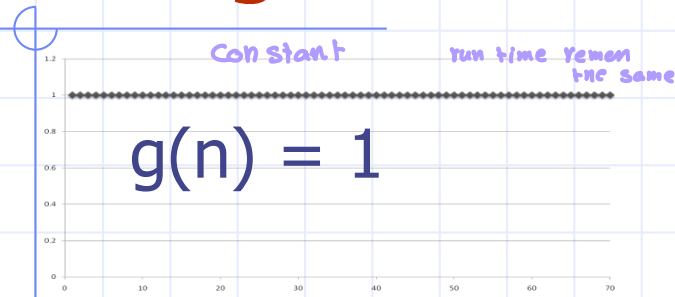
1E+30
1E+27
1E+24
1E+21
1E+18
1E+15
1E+12
1E+9
1E+6
1E+3
1E+0



- In a log-log chart, the slope of the line corresponds to the growth rate

Functions Graphed Using "Normal" Scale

Slide by Matt Stallmann included with permission.



Common order-of-growth classifications

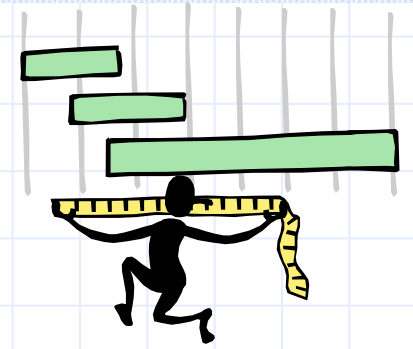
* loop / 2
or
قسمه
له خرب
→ log N

* الوباء المتزايدة (تتبعها)
الأكواد المتتابعة (تتبعها).

order of growth	name	typical code framework	description	example	$T(2N) / T(N)$
1	constant	$a = b + c;$	statement	add two numbers	1
$\log N$	logarithmic	$\text{while } (N > 1)$ $\{ N = N / 2; \dots \}$	divide in half	binary search	~ 1
N	linear	$\text{for } (\text{int } i = 0; i < N; i++)$ $\{ \dots \}$	loop	find the maximum	2
$N \log N$	linearithmic	[see mergesort lecture]	divide and conquer	mergesort	~ 2
N^2	quadratic	$\text{for } (\text{int } i = 0; i < N; i++)$ $\text{for } (\text{int } j = 0; j < N; j++)$ $\{ \dots \}$	double loop	check all pairs	4
N^3	cubic	$\text{for } (\text{int } i = 0; i < N; i++)$ $\text{for } (\text{int } j = 0; j < N; j++)$ $\text{for } (\text{int } k = 0; k < N; k++)$ $\{ \dots \}$	triple loop	check all triples	8
2^N	exponential	[see combinatorial search lecture]	exhaustive search	check all subsets	$T(N)$

المحور وقت ممكن

Estimating Running Time



- Algorithm **arrayMax** executes $5n + 5$ primitive operations in the worst case, $4n + 5$ in the best case. Define: _____

a = Time taken by the fastest primitive operation

b = Time taken by the **slowest** primitive operation

- Let $T(n)$ be **worst-case** time of **arrayMax**. Then

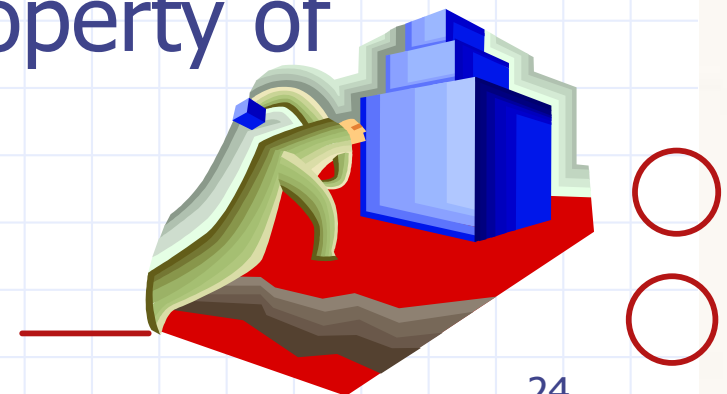
$$\underline{a(4n + 5)} \leq \underline{T(n)} \leq \underline{b(5n + 5)}$$

- Hence, the running time $T(n)$ is bounded by two linear functions

Growth Rate of Running Time

- Changing the hardware/ software environment
 - Affects $T(n)$ by a constant factor, but
 - Does not alter the growth rate of $T(n)$
- The linear growth rate of the running time $T(n)$ is an intrinsic property of algorithm arrayMax

arrayMax : $5 + 7n$
↳ Linear



Why Growth Rate Matters

if runtime is...	time for $n + 1$	time for $2n$	time for $4n$
$c \lg n$	$c \lg (n + 1)$	$c (\lg n + 1)$	$c(\lg n + 2)$
cn	$c(n + 1)$	$2cn$	$4cn$
$cn \lg n$	$\sim cn \lg n + cn$	$2cn \lg n + 2cn$	$4cn \lg n + 4cn$
cn^2	$\sim cn^2 + 2cn$	$4cn^2$	$16cn^2$
cn^3	$\sim cn^3 + 3cn^2$	$8cn^3$	$64cn^3$
$c2^n$	$c2^{n+1}$	$c2^{2n}$	$c2^{4n}$

A cows $\leftarrow$ $\rightarrow$ grow

runtime quadruples when problem size doubles

Practical implications of order-of-growth

from top
to bottom
inc size

growth rate	problem size solvable in minutes			
	1970s	1980s	1990s	2000s
1	any	any	any	any
log N	any	any	any	any
N	millions	tens of millions	hundreds of millions	billions
N log N	hundreds of thousands	millions	millions	hundreds of millions
N ²	hundreds	thousand	thousands	tens of thousands
N ³	hundred	hundreds	thousand	thousands
2 ^N	20	20s	20s	30

$$T(n) = 2 + 2 + n + 1 + 2n + 2n = 5 + 7n$$

Bottom line. Need linear or linearithmic alg to keep pace with Moore's law.

Practical implications of order-of-growth

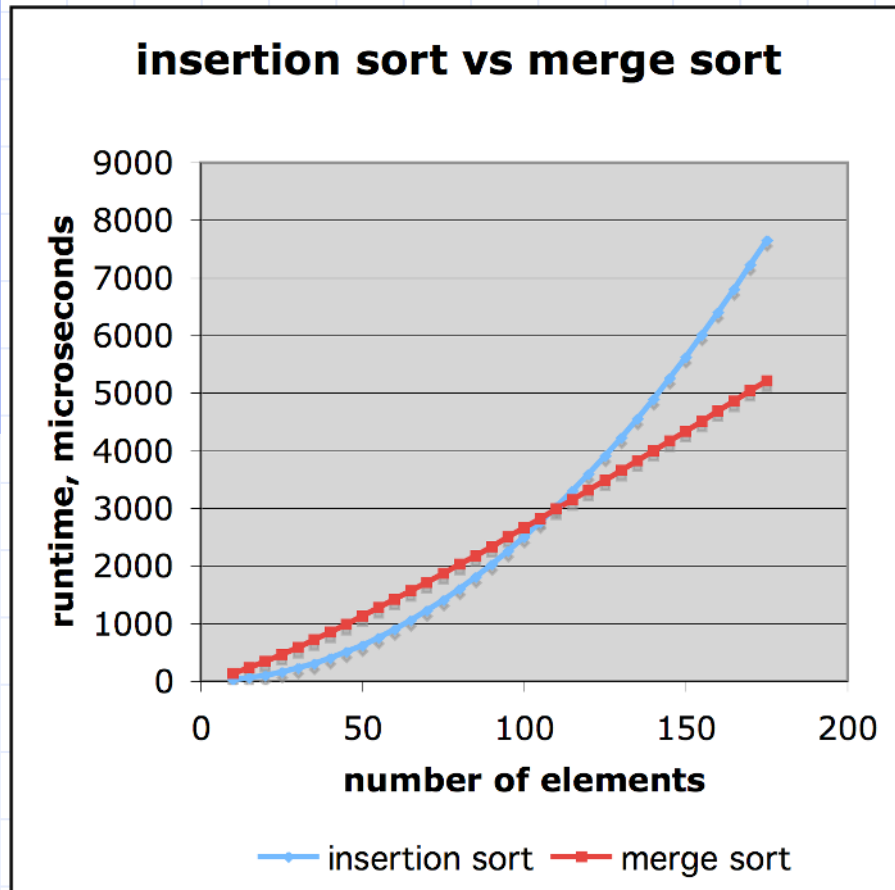
growth rate	problem size solvable in minutes				time to process millions of inputs			
	1970s	1980s	1990s	2000s	1970s	1980s	1990s	2000s
1	any	any	any	any	instant	instant	instant	instant
log N	any	any	any	any	instant	instant	instant	instant
N	millions	tens of millions	hundreds of millions	billions	minutes	seconds	second	instant
N log N	hundreds of thousands	millions	millions	hundreds of millions	hour	minutes	tens of seconds	seconds
N ²	hundreds	<u>thousand</u>	thousands	tens of thousands	decades	years	months	weeks
N ³	hundred	hundreds	thousand	thousands	never	never	never	millennia

Run time Yenan the Same.

Practical implications of order-of-growth

growth rate	name	description	effect on a program that runs for a few seconds	
			time for 100x more data	size for 100x faster computer
1	constant	independent of input size	–	–
$\log N$	logarithmic	nearly independent of input size	–	–
N	linear	optimal for N inputs	a few minutes	100x
$N \log N$	linearithmic	nearly optimal for N inputs	a few minutes	100x
N^2	quadratic	not practical for large problems	several hours	10x
N^3	cubic	not practical for medium problems	several weeks	4–5x
2^N	exponential	useful only for tiny problems	forever	1x

Comparison of Two Algorithms



insert $\rightarrow n^2$

merge sort $\rightarrow n \log_2 n$

insertion sort is
 $n^2 / 4$

merge sort is $\rightarrow$ *recursion* *ریع*
 $2 n \lg n$

sort a million items?

insertion sort takes
roughly **70 hours**

while

merge sort takes
roughly **40 seconds**

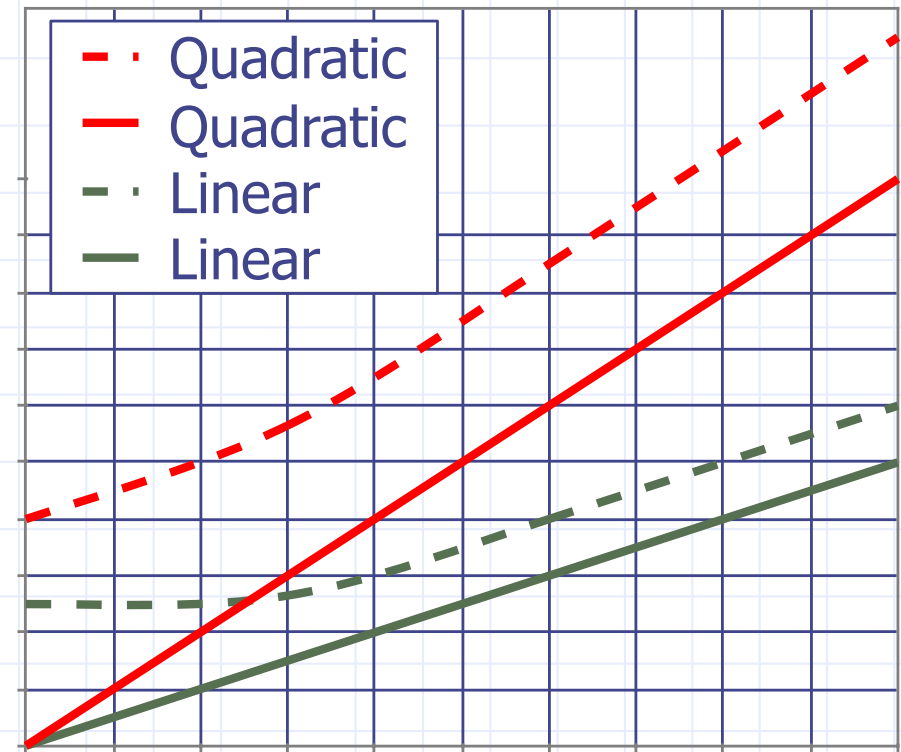
This is a slow machine, but if
100 x as fast then it's **40 minutes**
versus less than **0.5 seconds**

Constant Factors

- The growth rate is not affected by
 - constant factors or
 - lower-order terms
- Examples
 - $10^2 n + 10^5$ is a linear function
 - $10^5 n^2 + 10^8 n$ is a quadratic function

$T(n)$

1E+20
1E+16
1E+12
1E+8
1E+4
1E+0



1E+0 1E+2 1E+4 1E+6 1E+8 1E+10

n

$$T(n) = 5 + 7n$$

Big-Oh Notation

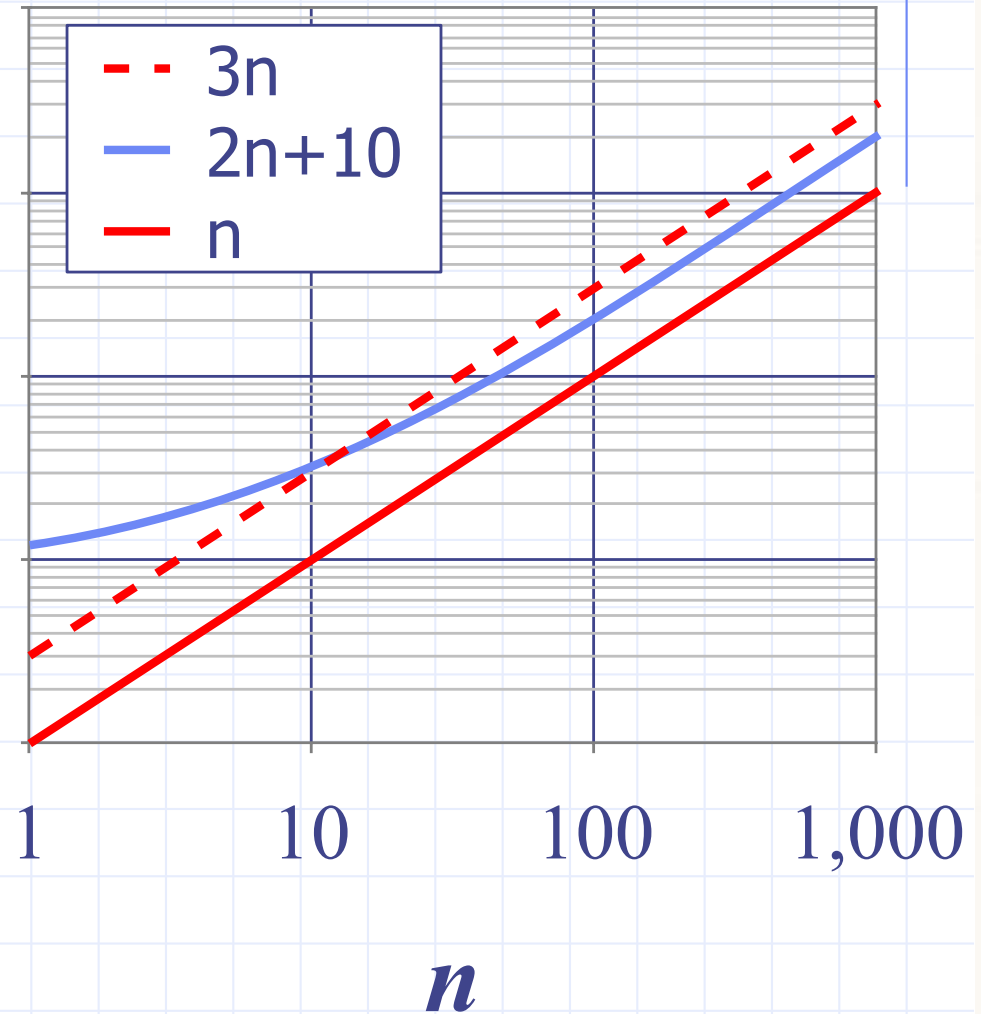
- Given functions $f(n)$ and $g(n)$, we say that $f(n)$ is $O(g(n))$ if there are positive constants

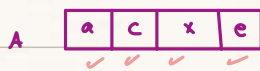
c and n_0 such that

$$f(n) \leq cg(n) \text{ for } n \geq n_0$$

- Example: $2n + 10$ is $O(n)$

- $2n + 10 \leq cn$
- $(c - 2)n \geq 10$
 - $n \geq 10/(c - 2)$
 - Pick $c = 3$ and $n_0 = 10$





- Search ('a') : 1 time $\rightarrow$ B.C.S

- Search ('x') : $\approx 1/2$ of the Input time $\rightarrow$ A.C.C

- Search ('y') : input size time $\rightarrow$ W.C.S

$f(n)$ is $O(g(n))$

$f(n) = 2n + 10$ $g(n) = n$

$f(n) \leq C \cdot g(n)$, $C > 1$
 $n \gg 1$

$2n + 10 \leq C \cdot n$

$10 \leq C \cdot n - 2n$

$10 \leq n(C - 2)$

$C = 3$ $10 \leq n$

or

$3n + 10 \leq Cn$ $C = 12$
 $n = 1$

$2n + 10 \leq 12n$

$2(1) + 10 \leq 12(1)$

$12 \leq 12$ ✓

Ex: if $f(n) = 3n + 2$ and $g(n) = n$

then Prove $f(n) = O(g(n))$?

$f(n) \leq C \cdot g(n)$

$3n + 2 \leq C \cdot n$
لـ زود واحد

$2n + 2 \leq 4n$
 $-3n$ $-3n$

$2 \leq 4n - 3n$

$2 \leq n$

this is true for $C=4$ and $n_0=2$

$$f(n) = n^2, g(n) = n$$

Big-Oh Example

- Example: the function

n^2 is not $O(n)$

- $n^2 \leq cn$
 $\rightarrow$ not constant
- $n \leq c$
 $\rightarrow$ some constant
- The above inequality cannot be satisfied since c must be a constant

1,000,000

100,000

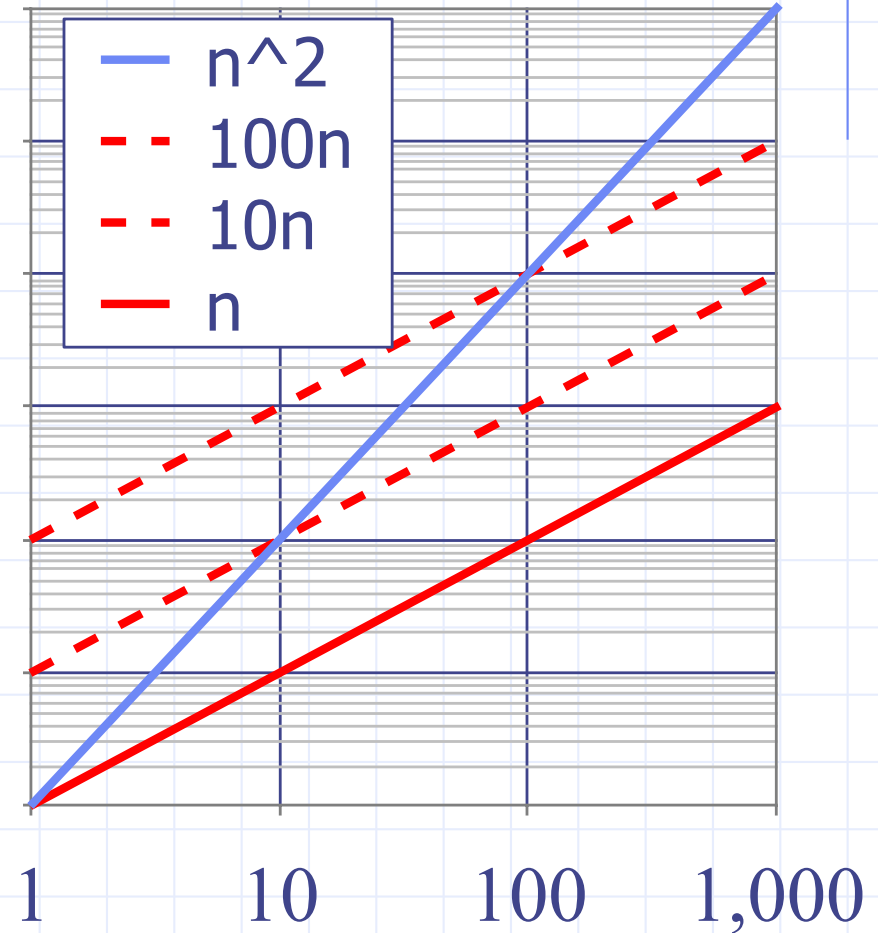
10,000

1,000

100

10

1



Constant or O(1) Complexity

- Example for Constant function O(1)

```
# O(1)
num = 10

def deductOne(num):
    num -= 1
    return num
```

```
# O(1)
num = 100

def deductOne(num):
    num -= 1
    return num
```

- Result

```
O(1):
9
```

```
O(1):
99
```

- Regardless of how big the input is, only one single operation is performed by algorithm that is num - 1

Link: <https://www.youtube.com/watch?v=0PPR5RWheFI>

34

الوقت ثابت لأن ما فية ١٥٥٢

Log-n or O(log n) Complexity

- Example for Constant function O(logn)

```
# O(log n)
num = 10

def divide(num):
    while num > 1:
        num /= 2
        print(num)
    return num

print "O(log n): "
divide(num)
```

```
# O(log n)
num = 100

def divide(num):
    while num > 1:
        num /= 2
        print(num)
    return num

print "O(log n): "
divide(num)
```

- Result

```
O(log n):
5
2
1
```

Time Complexity is 3

```
O(log n):
50
25
12
6
3
1
```

Time Complexity is 6

Link: <https://www.youtube.com/watch?v=0PPR5RWheFI>

35

لازم نشوف اول ال Counter نشوف التغير

ال ١٥٥٢ اللي فيها خرب او خمسة

$\log_2 n$

Example for Constant function $O(n)$

↓
۵ تا ۱۰ بار times

۵۰ پیف ۱۵ times

```
O(n):
[1]
[1, 1]
[1, 1, 1]
[1, 1, 1, 1]
[1, 1, 1, 1, 1]
[1, 1, 1, 1, 1, 1]
[1, 1, 1, 1, 1, 1, 1]
[1, 1, 1, 1, 1, 1, 1, 1]
[1, 1, 1, 1, 1, 1, 1, 1, 1]
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

Following are more complex:

$O(n \log n)$ ~ Complexity

- Quadratic $O(n^2) / O(n * n)$ - Complexity
- Cubic $O(n * n * n)$ - Complexity

- Cubic $O(n^3)$ Complexity
- Exponential $O(n^n)$ Complexity

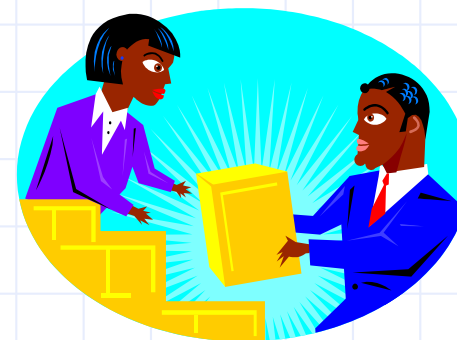
be.com/watch?v=DPR5RWwhoE

Link: <https://www.youtube.com/watch?v=DPPR5BWhe5I>

36

ال ۱۵۵۲ بعمل ملینا اتریقن .

More Big-Oh Examples



□ $7n - 2$

$7n - 2$ is $O(n)$

$$7n - 2 \leq cn$$

$$7n - 2 \leq 8n \rightarrow -2 \leq 8n - 7n \rightarrow -2 \leq n$$

need $c > 0$ and $n_0 \geq 1$ such that $7n - 2 \leq cn$ for $n \geq n_0$

this is true for $c = 7$ and $n_0 = 1$

□ ~~$3n^3 + 20n^2 + 5$~~

$3n^3 + 20n^2 + 5$ is $O(n^3)$

$$3n^3 + 20n^2 + 5 \leq cn^3$$

$$20n^2 + 5 \leq 4n^3 - 3n^2$$

need $c > 0$ and $n_0 \geq 1$ such that $3n^3 + 20n^2 + 5 \leq cn^3$ for $n \geq n_0$

this is true for $c = 4$ and $n_0 = 21$

□ $3 \log n + 5$

$3 \log n + 5$ is $O(\log n)$

$$3 \log n + 5 \leq 8 \log n$$

$$8 \leq 8 \log n, \quad 1 \leq \log n$$

$$n \geq 2$$

need $c > 0$ and $n_0 \geq 1$ such that $3 \log n + 5 \leq c \log n$ for $n \geq n_0$

this is true for $c = 8$ and $n_0 = 2$

← عندما نزيد

لاختيار (c) ما

ازيد اولاً رقم او

اجمع كل الارقام

الموجودة.

$$\frac{3n^3 + 20n^2 + 5}{f(n)} \text{ is } o\left(\frac{n^3}{g(n)}\right)$$

$$c > 1$$

$$n \geq 1$$

$$f(n) \leq c(g(n))$$

$$3n^3 + 20n^2 + 5 \leq cn^3$$

$$5 \leq cn^3 - 3n^3 - 20n^2$$

$$5 \leq n^3(c - 3) - 20n^2$$

$$c = 4$$

$$\frac{5 \leq n^3 - 20n^2}{\text{true}}$$

$$n=1 \rightarrow 5 \leq (1)^3 - 20(1)^2$$

$$5 \leq -19 \times$$

$$n=2 \rightarrow 5 \leq (2)^3 - 20(2)^2$$

$$8 - 80$$

$$n=10 \rightarrow 5 \leq (10)^3 - 20(10)^2$$

$$1000 - 2000$$

$$5 \leq -1000$$

$$n=20 \rightarrow 5 \leq 4000 - 20(20)^2$$

$$5 \leq 4000 - 8000$$

$$3 \log_2 n + 5 \text{ is } o(\log_2 n)$$

$$* 2^3 = 8 \rightarrow \log_2 8 = 3$$

$$f(n) = 3 \log_2 n + 5, g(n) = \log_2$$

$$f(n) \leq c(g(n))$$

$$f(n) \leq c \cdot (g(n))$$

Example: Find big-oh

$$16^5 n^4 + 16^8 n$$

$$n^4 + n = O(n^4)$$

$$10n^2 + 4n + 2 = O(n^2)$$

$$f(n) \leq c \cdot g(n)$$

$$10n^2 + 4n + 2 \leq cn^2$$

لەبەشود وایه

$$10n^2 + 4n + 2 \leq 11n^2$$

$$4n + 2 \leq 11n^2 - 10n^2$$

$$4n + 2 \leq n^2$$

this true for $n_0 = 5$ call, $n \geq 5$

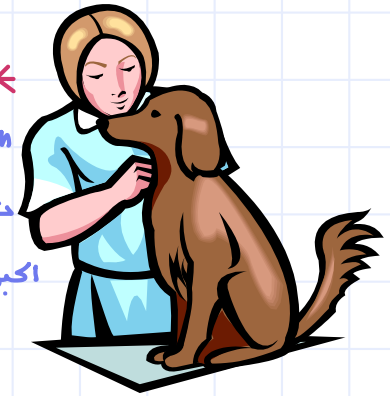
Big-Oh and Growth Rate

- The big-Oh notation gives an worse case upper bound on the growth rate of a function
- The statement " $f(n)$ is $O(g(n))$ " means that the growth rate of $f(n)$ is no more than the growth rate of $g(n)$ عزف يكون $f(n)$ $\leq$
- We can use the big-Oh notation to rank functions according to their growth rate في الرتبة ال Growth rate
اقل من n
الي فوق n
والتي تحت ؟

	$f(n)$ is $O(g(n))$	$g(n)$ is $O(f(n))$
$g(n)$ grows ^{faster} more	Yes	No
$f(n)$ grows more	No	Yes ✓
Same growth	Yes	Yes

Big-Oh Rules

* معلومه مهمة:
 Oh-لازم يكون صرح من الحدود
 متساوي واحد من الموجودين في المعادله او
 اكبر منه.



→ أكثر من حد

- If is $f(n)$ a polynomial of degree d , then $f(n)$ is $O(n^d)$, i.e.,

* القاعدة بتقول:

1. Drop lower-order terms → الحدود الصغيرة
 بـجعلها.
2. Drop constant factors → وناخذ الحد الكبير

إذا اخترت function من هون وهو (n) .

وقلت الـ $O(n)$ بتقبل. بس العكس مش صحيح

اختر (n) واعطيك حد اجزمه واقول هذا

Oh-لازم غلط

لازم يساويه او اكبر منه

ويفضل دائم يساويه.

- Use the smallest possible class of functions

- Say " $2n$ is $O(n)$ " instead of " $2n$ is $O(n^2)$ "

- Use the simplest expression of the class

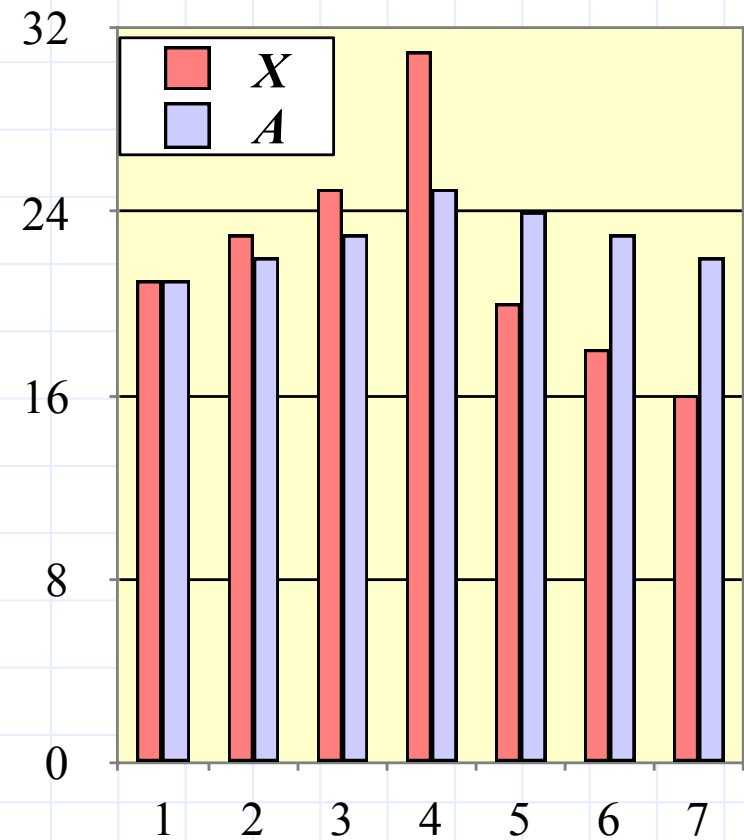
- Say " $3n + 5$ is $O(n)$ " instead of " $3n + 5$ is $O(3n)$ "

Asymptotic Algorithm Analysis

- The asymptotic analysis of an algorithm determines the running time in **big-Oh notation**
- To perform the asymptotic analysis
 - We find the **worst-case** number of primitive operations executed as a function of the input size
 - We express this function with big-Oh notation
- Example:
 - We say that algorithm **arrayMax** "runs in **$O(n)$ time"**
- Since constant factors and lower-order terms are eventually dropped anyhow, we can disregard them when counting primitive operations

Computing Prefix Averages

- We further illustrate asymptotic analysis with two algorithms for prefix averages
- The i -th prefix average of an array X is average of the first $(i + 1)$ elements of X :
$$A[i] = (X[0] + X[1] + \dots + X[i]) / (i+1)$$
- Computing the array A of prefix averages of another array X has applications to financial analysis



Prefix Averages (Quadratic)

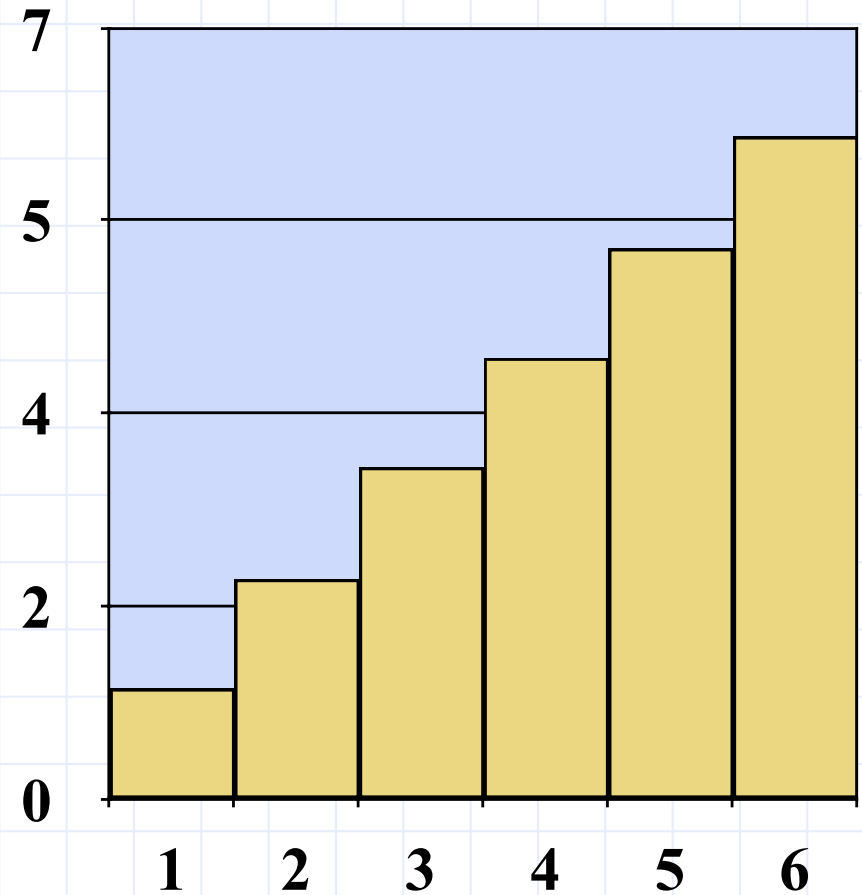
The following algorithm computes prefix averages in quadratic time by applying the definition

```
1  /** Returns an array a such that, for all j, a[j] equals the average of x[0], ..., x[j]. */
2  public static double[] prefixAverage1(double[] x) {
3      int n = x.length; ← C haven once
4      double[] a = new double[n]; C · n // filled with zeros by default
5      for (int j=0; j < n; j++) { → n
6          double total = 0; → C · n // begin computing x[0] + ... + x[j]
7          for (int i=0; i <= j; i++) n × n = n²
8              total += x[i]; → C × n²
9              a[j] = total / (j+1); C × n // record the average
10         }
11     return a;
12 }
```

$$T(n) = C_1 n + n + 2C_2 n^2 + n^2 + 1 \rightarrow O(n^2)$$

Arithmetic Progression

- The running time of **prefixAverage1** is $O(1 + 2 + \dots + n)$
- The sum of the first n integers is $n(n + 1) / 2$
 - There is a simple visual proof of this fact
- Thus, algorithm **prefixAverage1** runs in $O(n^2)$ time



Prefix Averages 2 (Linear)

The following algorithm uses a running summation to improve the efficiency

```
1  /** Returns an array a such that, for all j, a[j] equals the average of x[0], ..., x[j]. */
2  public static double[] prefixAverage2(double[] x) {
3      int n = x.length; c
4      double[] a = new double[n]; c n or n // filled with zeros by default
5      double total = 0; c // compute prefix sum as x[0] + x[1] + ...
6      for (int j=0; j < n; j++) { n
7          total += x[j]; c n // update prefix sum to include x[j]
8          a[j] = total / (j+1); c * n = c n // compute average based on current sum
9      }
10     return a; c
11 }
```

$T(n) = 2c + 2cn + 2cn \rightarrow O(n)$

Algorithm **prefixAverage2** runs in $O(n)$ time!

Math you need to Review



- Summations
- Powers
- Logarithms
- Proof techniques
- Basic probability

□ Properties of powers:

$$a^{(b+c)} = a^b a^c$$

$$a^{bc} = (a^b)^c$$

$$a^b / a^c = a^{(b-c)}$$

$$b = a^{\log_a b}$$

$$b^c = a^{c \cdot \log_a b}$$

□ Properties of logarithms:

$$\log_b(xy) = \log_b x + \log_b y$$

$$\log_b(x/y) = \log_b x - \log_b y$$

$$\log_b x a = a \log_b x$$

$$\log_b a = \log_x a / \log_x b$$

Relatives of Big-Oh



big-Omega

- $f(n)$ is $\Omega(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that

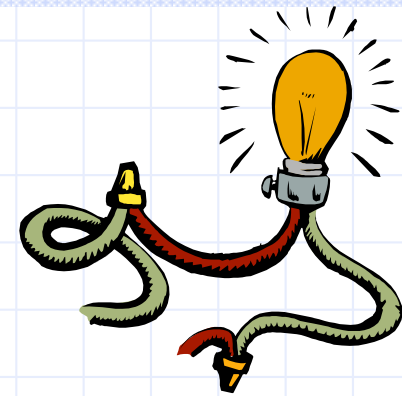
$$f(n) \geq c g(n) \text{ for } n \geq n_0$$

big-Theta

- $f(n)$ is $\Theta(g(n))$ if there are constants $c' > 0$ and $c'' > 0$ and an integer constant $n_0 \geq 1$ such that

$$c'g(n) \leq f(n) \leq c''g(n) \text{ for } n \geq n_0$$

Intuition for Asymptotic Notation



big-Oh

- $f(n)$ is $O(g(n))$ if $f(n)$ is asymptotically **less than or equal to** $g(n)$

big-Omega

$$f(n) \geq c \cdot g(n)$$

- $f(n)$ is $\Omega(g(n))$ if $f(n)$ is asymptotically **greater than or equal to** $g(n)$

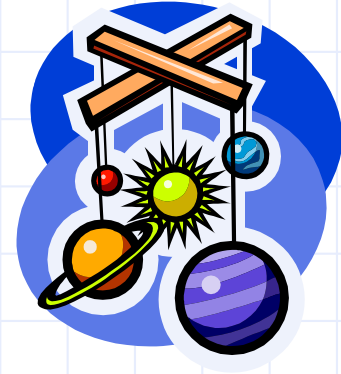
big-Theta

- $f(n)$ is $\Theta(g(n))$ if $f(n)$ is asymptotically **equal to** $g(n)$

$$c_1 \cdot g(n) < f(n) < c_2 \cdot g(n)$$

from the same class

Example Uses of the Relatives of Big-Oh



- $5n^2$ is $\Omega(n^2)$

$$\begin{array}{l} 5n^2 \geq cn^2 \\ 5n^2 \geq 6n^2 \quad | \quad 1 \geq 6n^2 - 5n^2 \end{array}$$

$f(n)$ is $\Omega(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \geq c g(n)$ for $n \geq n_0$

let $c = 5$ and $n_0 = 1$

- $5n^2$ is $\Omega(n)$

$$5n^2 \geq cn$$

$$5 \geq 1 \quad \checkmark$$

$f(n)$ is $\Omega(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \geq c g(n)$ for $n \geq n_0$

let $c = 1$ and $n_0 = 1$

- $5n^2$ is $\Theta(n^2)$

$$5n^2 \leq cn^2$$

$f(n)$ is $\Theta(g(n))$ if it is $\Omega(n^2)$ and $O(n^2)$. We have already seen the former, for the latter recall that $f(n)$ is $O(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \leq c g(n)$ for $n \geq n_0$

Let $c = 5$ and $n_0 = 1$

$\rightarrow \log n$ වැ

$$2n(\log_2 n - 1)$$

In general, we should use the big-Oh notation to characterize a function as closely as possible. While it is true that the function $f(n) = 4n^3 + 3n^2$ is $O(n^5)$ or even $O(n^4)$, it is more accurate to say that $f(n)$ is $O(n^3)$. Consider, by way of analogy,

النهاية - البداية + ١

$n - 0 + 1$

```

public static int example5(int [] first, int [] second)
{
    int n = first.length, count = 0;
    for (int i = 0; i < n; i++) {
        int total = 0;
        for (int j = 0; j < n; j++) {
            for (int k = 0; k <= j; k++) {
                total += first[k];
                if (second[i] == total) count++;
            }
        }
    }
    return count;
}

```

2, 3 → لأن بحسبنا نكتب
 .deder
 $i = i + 1 \rightarrow 2 \times n$
 $j = 0 \rightarrow 2 \times n$
 $k = 0 \rightarrow 2 \times n \times n$
 $n(n+1)$
 $n(n+1)$
 $4n$

$O(n^3)$

depend loop:

$j = 0 ; j < n$

$k = 0 ; k <= j$

n = 4	K	
j		
0	0	stop
1	0 1	1
2	0 1 2	2
3	0 1 2 3	3
4 → stop		4

إذا كان اللوب الى جوار يعتمد على اللوب الى جوار

عدد تكرارات اللوب الى جوار وجوده في اللوب الى جوار

$\frac{n(n+1)}{2}$

$\frac{4(4+1)}{4}$

$$* (n-2) + (n-1) + n$$



$$n^2$$

$$\frac{7n^2 + 3n \log_2 n}{n} \leq C \cdot n^2$$

$$7n + 3 \log_2 n \leq C \cdot n^2$$

$$n=1 \quad 7(1) + 3(0) \leq (1)^2 \cdot x$$

$$n=1 \quad 7(2) + 3(1) \leq (2)^2 \cdot x$$

$$n=2 \quad 7(4) + 3(2) \leq (4)^2 \cdot x$$

$$n=4 \quad 7(8) + 3(3) \leq (8)^2 \rightarrow 65 \leq 64$$

$$n=8 \quad 7(16) + 3(4) \leq (16)^2$$

$$n=16$$